

AI RAN Demo 算法详解文档

版本: Alpha 1.0

文档定位: 面向通信知识入门读者，系统性地介绍AI自演进RAN原型中涉及的所有关键算法

覆盖范围: 通信基础概念 → 链路自适应 → KPI估算 → 信道模型 → 数字孪生验证 → 自演进优化 → 波形与频谱

目录

- 第一部分：通信基础概念速览
- 第二部分：通信模块算法
 - 2.1 调制模块
 - 2.2 信道编码模块
 - 2.3 HARQ 混合自动重传请求模块
 - 2.4 导频模块
 - 2.5 调度器模块
 - 2.6 功率控制模块
 - 2.7 安全回退模块
- 第三部分：链路自适应算法
- 第四部分：KPI 估算模型
- 第五部分：信道模型算法
- 第六部分：数字孪生验证算法
 - 6.1 接口校验算法
 - 6.2 约束校验算法
 - 6.3 链路级仿真算法
 - 6.4 故障注入算法
 - 6.5 综合评分算法
 - 6.6 沙盒决策算法
- 第七部分：自演进与优化算法
 - 7.1 进化循环算法
 - 7.2 启发式优化推荐算法
 - 7.3 历史策略推荐算法
- 第八部分：频段配置算法
- 第九部分：波形生成与频谱分析算法

- 9.1 IQ 符号生成算法
- 9.2 AWGN 噪声叠加算法
- 9.3 功率谱密度估计算法
- 9.4 BER 估算算法
- 第十部分：实验运行器算法
- 附录：算法索引表

第一部分：通信基础概念速览

在深入算法之前，先理解以下几个无线通信的核心概念，后续所有算法都建立在这些基础之上。

概念	通俗解释	在系统中的含义
SNR (信噪比)	信号强度 与 背景噪声强度 的比值	衡量"信号有多清晰"。SNR越高，通信质量越好。单位：dB（分贝）
SINR (信干噪比)	信号强度 与 （干扰+噪声）强度 的比值	比SNR更真实——考虑了其他通信设备的干扰。SINR是做出链路决策的关键依据
BLER (误块率)	传输数据块中"传错了"的比例	例如BLER=0.1，意味着10%的数据块需要重传。这是衡量可靠性的核心指标
Throughput (吞吐量)	单位时间内成功传输的数据量	你的"网速"。单位：kbps（千比特每秒）
Latency (时延)	数据从发送到接收的延迟时间	例如视频通话中的卡顿延迟。单位：ms（毫秒）
Modulation (调制)	如何把数字比特"搭载"到无线电波上	就像"打包"数据——高阶调制一次打包更多比特，但抗干扰更弱
Coding Rate (编码率)	有用数据 / 总传输数据的比例	例如编码率0.5，意味着每2比特中有1比特是冗余校验信息
HARQ (混合自动重传)	传错了就自动重传的机制	像"没听清？再说一遍"——保证数据最终正确到达
MCS (调制编码策略)	调制方式 + 编码率 的组合等级	从0到28的索引，数字越大代表更"激进"的通信配置
Pilot (导频信号)	通信双方约定好的"训练信号"	像高速公路上的路标——接收端用来"校准"信道估计
Bandwidth (带宽)	可用的频谱宽度	带宽越大，能传的数据越多。我们的国道/高速公路宽度

第二部分：通信模块算法

系统的通信功能由一组可插拔的模块组成。每个模块封装一种通信能力，以统一接口被内核加载和执行。

2.1 调制模块

代码位置: [uran/modules/modulation.py](file:///e:/AI RAN Demo/uran/modules/modulation.py)

算法功能

调制模块负责决定"每个无线电符号携带多少个比特"。不同的调制方式代表了**速率与可靠性的权衡**——阶数越高，速率越大，但抗干扰能力越弱。

输入

参数	类型	说明
<code>modulation</code>	str	调制方式名称：BPSK / QPSK / 16QAM / 64QAM
<code>bits_per_symbol</code>	int	每符号携带比特数
<code>robustness</code>	float	鲁棒性系数（0~1，越大抗干扰越强）

输出

参数	类型	说明
<code>modulation</code>	str	选定的调制方式
<code>bits_per_symbol</code>	int	每符号比特数
<code>modulation_robustness</code>	float	鲁棒性系数

核心实现思路

模块采用**策略模式**设计。每种调制方式在系统中被预定义为独立的模块实例（如 `mod_bpsk`、`mod_qpsk`），注册到模块注册表中。当AI生成方案时，指定使用哪个调制模块，内核加载对应模块并写入运行时上下文。

系统支持的四种调制方式的定量关系：

调制方式	每符号比特数	所需最低信噪比	鲁棒性	适用场景
BPSK	1 bit	< -2 dB	1.0（最强）	极远距离、高可靠性
QPSK	2 bits	> 3 dB	0.8	均衡距离、中等速率
16QAM	4 bits	> 9 dB	0.5	近距离、较高吞吐
64QAM	6 bits	> 15 dB	0.3	极近距离、最高吞吐

通俗理解: BPSK 就像"每个包裹只装1件物品"——慢但安全。64QAM 就像"每个包裹装6件物品"——快但怕颠簸。

2.2 信道编码模块

代码位置: [uran/modules/coding.py](file:///e:/AI RAN Demo/uran/modules/coding.py)

算法功能

信道编码在发送的数据中加入**冗余信息**，使得接收端即使部分比特接收错误，也能自动检测甚至纠正错误。这是一种"用额外数据换取可靠性"的技术。

输入

参数	类型	说明
module_id	str	编码模块唯一标识
coding_rate	float	编码率（有效数据/总数据）
coding_gain_db	float	编码增益（dB），衡量纠错能力

输出

参数	类型	说明
coding_rate	float	编码率
coding_gain_db	float	编码增益

核心实现思路

系统支持三类编码方案，基于通信理论预设编码增益和时延参数：

编码方案	编码率	编码增益	处理时延	特点
重复码 (Rep 1/3)	0.33	5 dB	15 ms	最简单——数据重复3次，多数投票纠错
重复码 (Rep 1/2)	0.50	3 dB	12 ms	数据重复2次
卷积码 (Conv 1/2)	0.50	3 dB	8 ms	经典纠错码，低成本可靠
卷积码 (Conv 2/3)	0.67	2 dB	6 ms	效率更高的卷积编码
LDPC (2/3)	0.67	4 dB	5 ms	现代高效编码，5G标准编码
LDPC (3/4)	0.75	3 dB	4 ms	最高效、最低冗余

通俗理解: 编码就像"寄包裹时额外放入一个校验清单"。LDPC 就像一份精密的校验清单——用最少的额外纸张提供最强的检查能力。

2.3 HARQ 混合自动重传请求模块

代码位置: [uran/modules/harq.py](file:///e:/AI RAN Demo/uran/modules/harq.py)

算法功能

HARQ 是无线通信中最重要的可靠性保障机制之一。当接收端发现数据传输有误，它请求发送端**重传**数据。系统支持从"无HARQ"到"增量冗余"HARQ的多级配置。

输入

参数	类型	说明
<code>module_id</code>	str	模块ID
<code>max_retx</code>	int	最大重传次数

输出

参数	类型	说明
harq_max_retx	int	最大重传次数
harq_gain_db	float	HARQ带来的有效信噪比增益 = max_retx × 1.2 dB
harq_latency_penalty_ms	float	HARQ带来的额外时延 = max_retx × 8 ms

核心实现思路

HARQ 的增益和代价由两个线性公式决定：

```
HARQ增益 (dB) = 最大重传次数 × 1.2
HARQ时延代价 (ms) = 最大重传次数 × 8
```

- **0次重传 (无HARQ)** : 0增益 / 0时延——最快但不纠错
- **1~2次重传**: 适合时延敏感场景
- **3~4次重传**: 适合高可靠性场景，但时延显著增加

通俗理解: HARQ 就像"没听清，请再说一遍"。设定 `max_retx=4` 表示最多请对方重复4次。重传越多，信息越准确，但耗时越长。

2.4 导频模块

代码位置: [uran/modules/pilot.py](file:///e:/AI RAN Demo/uran/modules/pilot.py)

算法功能

导频信号 (Pilot) 是收发双方约定的"已知训练信号"。接收端通过分析导频信号的畸变来**估计信道状态** (信号经历了怎样的衰减和畸变)。导频密度越高，信道估计越准确，但占用越多频谱资源。

输入

参数	类型	说明
density	str	导频密度等级: low / medium / high

输出

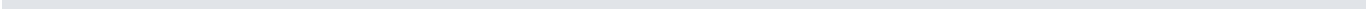
参数	类型	说明
pilot_density	str	导频密度级别

核心实现思路

三档导频密度的设计参数：

密度等级	信道估计增益	频谱效率系数	适用场景
high	+2 dB	75%	弱信号、高移动性、抗干扰场景
medium	+1 dB	88%	一般信道条件
low	0 dB	95%	强信号、低成本、高吞吐场景

通俗理解: 导频就像高速公路上每隔一段就有的"路标"。路标越多（高密度），你在雾天也能看清路；但路标也占用了本可以行驶的道路空间。



2.5 调度器模块

代码位置: [uran/modules/scheduler.py](file:///e:/AI RAN Demo/uran/modules/scheduler.py)

算法功能

调度器决定"以什么策略在多个通信任务之间分配无线资源"。不同的调度策略反映了**公平性、吞吐、可靠性、时延**之间的权衡。

输入

参数	类型	说明
scheduler_type	str	调度策略类型

输出

参数	类型	说明
scheduler	str	调度策略名称

核心实现思路

调度策略	调度时延	设计理念
throughput_first	5 ms	优先分配资源给信道质量最好的用户——"强者多得"
balanced	4 ms	在吞吐和公平之间折中
reliability_first	10 ms	优先保障弱信号用户的可靠性
latency_first	0 ms	优先调度时延敏感的数据——"急事先办"
fairness	6 ms	轮询式的公平分配——"人人有份"

通俗理解: 调度器就像机场塔台——决定哪架飞机先降落。 `throughput_first` 优先让大飞机（高吞吐用户）降落； `latency_first` 优先让急救飞机（时延敏感任务）降落。

2.6 功率控制模块

代码位置: [uran/modules/power_control.py](file:///e:/AI RAN Demo/uran/modules/power_control.py)

算法功能

功率控制决定发射功率的调节策略。发射功率直接决定了信号可覆盖的距离和抵抗干扰的能力，同时功耗是移动设备续航的关键因素。

输入 (与输出同名)

参数	类型	说明
<code>control_type</code>	str	功控策略名称

核心实现思路

功控策略	设计理念
fixed	固定功率不变——最简单，但无法适应信道变化
snr_based	根据信号质量动态调整——"信号弱就喊大声点"
conservative	偏向使用较高功率以保障可靠性
aggressive	偏向使用较低功率以节省能量（风险较高）
energy_saving	以最小功耗为首要目标——适合IoT、卫星场景

2.7 安全回退模块

代码位置: [uran/modules/fallback.py](file:///e:/AI RAN Demo/uran/modules/fallback.py)

算法功能

安全回退模块是系统的**最后防线**。当运行时监控到系统工作异常（如连续的链路失败、AI方案实际表现不佳），系统会触发回退机制，强制切换到一套**已知安全的保守配置**。

输入

无特殊输入——被触发时自动生效。

输出

- 调制 = BPSK（最鲁棒）
- 编码率 = 0.33（最高冗余）
- HARQ = 4次最大重传
- 导频密度 = high
- 功控 = conservative

核心实现思路

回退配置设计为“极端保守方案”：牺牲速度和效率，换取最大化的可靠性。这是一种**安全设计模式**，在航空、自动驾驶等安全关键系统中广泛应用。

通俗理解: 回退就像汽车的“备胎”——不是最优解，但当你爆胎时总能带你安全到达目的地。系统检测到异常时自动切到“安全模式”。

第三部分：链路自适应算法

代码位置: [uran/sim/adaptation.py](file:///e:/AI RAN Demo/uran/sim/adaptation.py)

链路自适应（Link Adaptation）是无线通信系统的**核心决策算法**。它根据实时信道质量，自动选择最优的通信参数组合。

算法功能

在无线通信中，信道质量是实时变化的。链路自适应算法回答一个问题：**在当前信道条件下，应该用什么样的调制、编码、功率等参数组合来传输数据？**

输入

参数	类型	说明
snr_db	float	当前信噪比（dB）
sinr_db	float None	当前信干噪比（dB），优先于SNR
previous_bler	float None	上一时刻的误块率（用于自适应惩罚）

输出（LinkAdaptationDecision）

参数	类型	说明
mcs	int	选定的MCS等级
modulation	str	选定的调制方式
coding	str	选定的编码方案
coding_rate	float	选定的编码率
pilot_density	float	选定的导频密度
harq_retx	int	选定的HARQ重传次数
tx_power_dbm	float	选定的发射功率（dBm）
reason	str	决策原因描述
event_level	str	事件级别（success/info/warning/critical）

核心实现思路

算法分两步执行：

第1步：计算有效信道质量

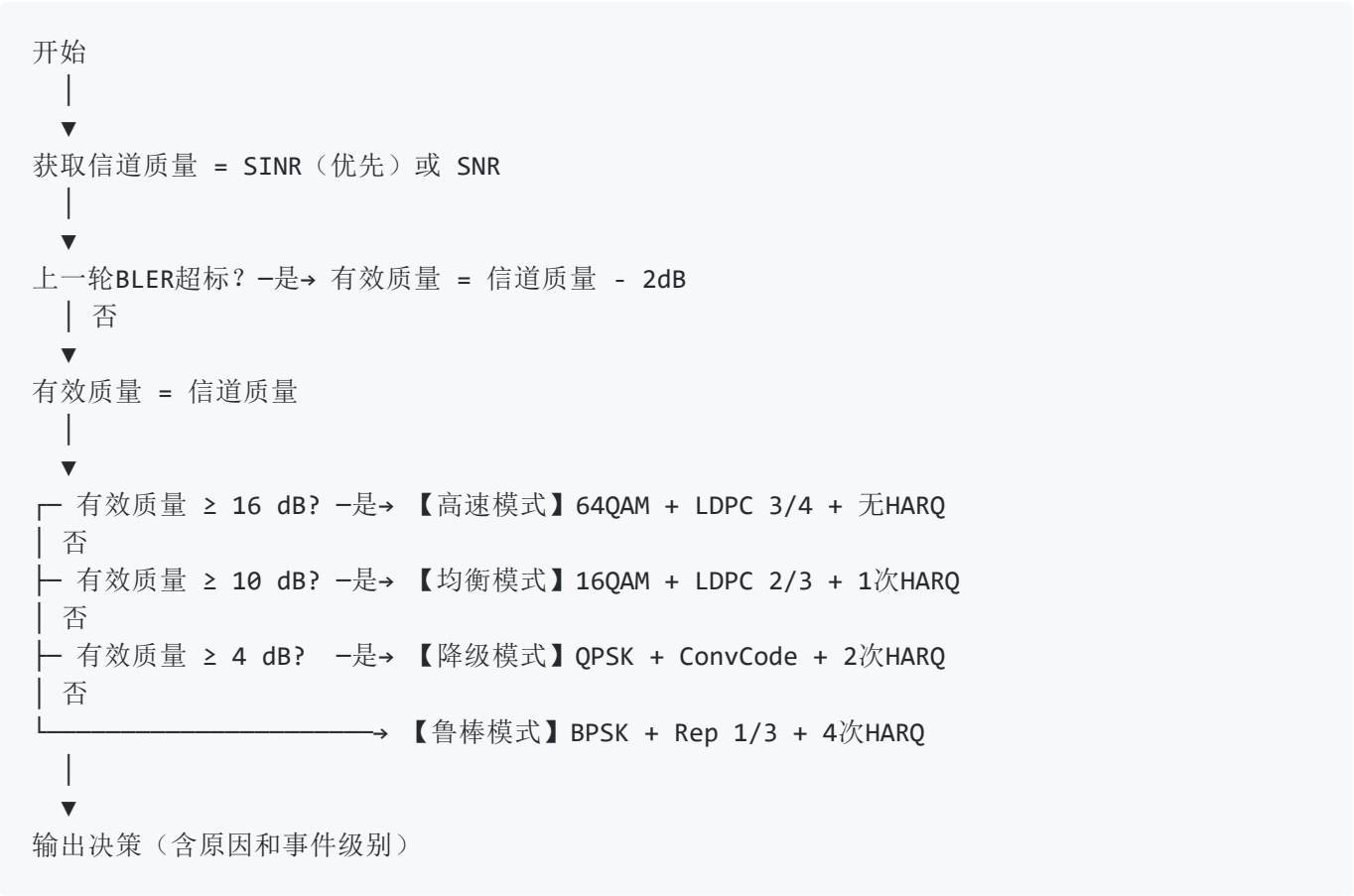
选择质量参考值 = SINR（如果有） 否则使用 SNR
BLER惩罚 = 如果上轮BLER > 目标BLER，扣 2 dB
有效质量 = 质量参考值 - BLER惩罚

这里的 BLER 惩罚设计很巧妙——如果上一轮已经出错很多了，说明当前策略可能不够鲁棒，需要更保守的决策。

第2步：四档决策表

有效质量	MCS	调制	编码	编 码 率	导频 密度	HARQ	发射 功率	决策级 别
≥ 16 dB	8	64QAM	LDPC	0.75	0.0625	0	14 dBm	☑ success
≥ 10 dB	6	16QAM	LDPC	0.66	0.125	1	16 dBm	📄 info
≥ 4 dB	3	QPSK	ConvCode	0.50	0.125	2	18 dBm	⚠ warning
< 4 dB	1	BPSK	RepetitionCode	0.33	0.25	4	20 dBm	⬤ critical

算法流程图



通俗理解: 链路自适应就像汽车的自适应巡航——路况好（高SINR）就加速（高MCS），路况差就减速（低MCS），前方的拥堵（高BLER）预示即将到来的路况变差，提前放慢速度。

第四部分：KPI 估算模型

代码位置: [uran/sim/kpi_model.py](file:///e:/AI RAN Demo/uran/sim/kpi_model.py)

KPI（关键性能指标）估算模型是系统**量化通信性能的数学核心**。它基于通信理论中的经典模型，将信道质量、调制编码参数"翻译"成可度量的 KPI 数值。

算法功能

输入一组通信参数和信道质量，估算该配置下的 BLER、吞吐量、时延、频谱效率、可靠性等关键指标。

输入

参数	类型	说明
snr_db	float	当前信噪比 (dB)
sinr_db	float	当前信干噪比 (dB)
modulation	str	调制方式
coding_rate	float	编码率
pilot_density	float	导频密度
harq_retx	int	HARQ 重传次数

输出

参数	类型	含义
bler	float	误块率 (Block Error Rate) , 0~1
throughput_kbps	float	吞吐量 (千比特每秒)
latency_ms	float	时延 (毫秒)
spectral_efficiency	float	频谱效率 (每Hz频谱传输的比特数)
packet_success_rate	float	包成功概率 = 1 - BLER
reliability	float	可靠性指标 (0~1)

核心实现思路

KPI 模型分为 5 个子计算步骤:

步骤 1: 计算有效 SINR

```
有效 SINR = SINR - 频率惩罚
```

频率越高，电磁波传播损耗越大。系统根据频段配置自动计算频率惩罚值（详见第八部分）。

步骤 2: Sigmoid BLER 模型

```
margin = 有效SINR - 调制所需最低SNR
BLER = 1 / (1 + e^(0.9 * margin))
```

其中:

- 调制所需最低SNR：BPSK=-2dB, QPSK=3dB, 16QAM=9dB, 64QAM=15dB
- 使用 Sigmoid 函数（S形曲线）模拟 BLER 随信噪比平滑过渡的物理特性
- BLER 被限制在 [0.0001, 0.99] 之间，避免极端值

为什么用 Sigmoid? 真实通信系统中，BLER 不会突然从0跳到1，而是随着信噪比降低呈平滑的S形增长。Sigmoid 函数很好地模拟了这一特点。

步骤 3: 吞吐量公式

频谱效率 = 调制阶数（每符号比特数） × 编码率

导频效率 = 1 - 导频密度

HARQ效率 = 1 / (1 + 0.25 × 重传次数)

包成功率 = 1 - BLER

吞吐量(kbps) = 带宽(kHz) × 频谱效率 × 导频效率 × HARQ效率 × 包成功率

这个公式体现了：**带宽是"水管粗细"**，**频谱效率是"水流速"**，**各项效率是"损耗"**，**最终吞吐量是"实际到达的水量"**。

步骤 4：时延公式

时延(ms) = 基础时延(10ms)
+ 4.0 × HARQ重传次数
+ 8.0 × (1 - 编码率)
+ 2.0 × 导频密度

三项时延来源：

- **HARQ时延**：每次重传增加4ms
- **编码处理时延**：低码率编码需要更多处理时间
- **导频开销**：导频占用时间

步骤 5：可靠性指标

可靠性 = 包成功率 = 1 - BLER

频段配置对 KPI 的影响

系统支持预设频段（n41/n79）和自定义频段，频段会影响：

1. **带宽**：n41/n79_100MHz → 100,000 kHz，大幅提升吞吐量基数
2. **频率惩罚**：频率越高，惩罚越大（详见第八部分），直接影响有效SINR

通俗理解：KPI 模型就像一个"通信性能计算器"。输入你的通信参数设置和当前环境，它就能估算出你的"网速"（吞吐量）、"延迟"（时延）、"丢包率"（BLER）等关键指标。

第五部分：信道模型算法

代码位置: [uran/sim/channel_timeline.py](file:///e:/AI RAN Demo/uran/sim/channel_timeline.py)

信道模型模拟真实世界中无线信道的时变特性——信号不会一直很好，它会因距离、障碍物、干扰等因素而变化。

算法功能

生成随时间（以step为单位）变化的 SNR/SINR 序列，模拟不同的无线传播环境。

输入

参数	类型	说明
mode	str	信道模式
initial_snr_db	float	初始 SNR （dB）
final_snr_db	float	最差 SNR （dB）
total_steps	int	总仿真步数
seed	int	随机种子（保证可复现）
noise_std_db	float	随机游走模式的噪声标准差

输出

单个时间步的 SNR/SINR 值。

核心实现思路：7种信道模式

模式 1：恒定信道 (constant)

```
SNR(step) = 初始SNR（不变）
```

最简单的模式，适合**基线测试**——当信道完全不变时，系统表现如何？

模式 2：线性衰落 (linear_fading)

```
ratio = step / total_steps
SNR(step) = 初始SNR + ratio × (最终SNR - 初始SNR)
```

模拟**距离渐增导致信号衰减**的场景，如UE逐渐远离基站。

模式 3：正弦波动 (sinusoidal)

```
中心值 = (初始SNR + 最终SNR) / 2
振幅 = |初始SNR - 最终SNR| / 2
SNR(step) = 中心值 + 振幅 × sin(2π × step / total_steps)
```

模拟**多径衰落**——信号因反射、折射而周期性波动。

模式 4：随机游走 (random_walk)

```
漂移项 = (最终SNR - 初始SNR) / 总步数
SNR(step) = SNR(step-1) + 漂移项 + 高斯噪声(标准差=noise_std_db)
SNR 限制在 [-5, 30] dB 之间
```

最接近真实环境的模式——不可预测的随机波动，叠加一个长期趋势。

模式 5：遮挡 (blockage)

```
前45%时间：SNR = 初始SNR（正常）
45%~70%时间：SNR = 最终SNR（被遮挡，急剧恶化）
70%后：SNR = (初始+最终)/2（部分恢复）
```

模拟**建筑物突然遮挡**的场景——信号骤降后部分恢复。

模式 6：干扰突增 (jamming_spike)

```
40%~65%时间：SNR = 最终SNR（被干扰）
其他时间：SNR = 初始SNR（正常）
```

模拟**电子对抗/恶意干扰**场景——出现一段时间的强力干扰。

模式 7：类切换信道 (handover_like)

```
前30%：稳定在初始SNR（服务小区信号强）
30%~55%：SNR 线性下降（离开当前小区）
55%~75%：稳定在最终SNR（信号最差时刻）
75%后：SNR 线性回升（接入新小区）
```

模拟**移动中的小区切换**——用户从一个基站的覆盖范围移动到另一个。

SINR 计算

$SINR(step) = SNR(step) - \text{干扰惩罚}$

其中干扰惩罚在有干扰源（如jammer节点）时为3dB，正常情况下为1dB。

第六部分：数字孪生验证算法

数字孪生沙盒是整个系统的**安全验证引擎**。在AI生成的方案被实际部署之前，它必须通过沙盒的五步验证流程。

代码位置: [uran/twin/sandbox.py](file:///e:/AI RAN Demo/uran/twin/sandbox.py)

AI方案 → [1.接口校验] → [2.约束校验] → [3.链路仿真] → [4.故障注入] → [5.评分决策] → ACCEPT/REJECT

6.1 接口校验算法

代码位置: [uran/twin/validators.py](file:///e:/AI RAN Demo/uran/twin/validators.py) —
InterfaceValidator

算法功能

检查AI生成的方案是否包含所有必需的字段，且字段值不为空。

输入

- plan: AI生成的通信方案

输出

- passed: 布尔值，是否通过
- errors: 缺失字段列表

核心实现思路

逐一检查以下12个必填字段是否存在且非空： modulation、coding_scheme、coding_rate、mcs、harq_enabled、harq_max_retx、pilot_density、tx_power_dbm、scheduler、power_control

这是最基础的一层检查——就像检查表单是否已完整填写。

6.2 约束校验算法

代码位置: [uran/twin/validators.py](file:///e:/AI RAN Demo/uran/twin/validators.py) — ConstraintValidator

算法功能

检查方案中的参数是否在合理的物理和技术约束范围内，并根据任务需求给出警告。

输入

- plan : 通信方案
- intent : 用户任务需求

输出

- passed : 是否通过
- errors : 硬约束违规列表（直接导致拒绝）
- warnings : 软约束警告列表（仅提醒）

核心实现思路：双层规则

硬约束（不通过则拒绝）： | 检查项 | 约束范围 | |-----|-----| | 调制方式 | 必须在 [BPSK, QPSK, 16QAM, 64QAM] 中 | | 编码方案 | 必须在预定义的7种方案中 | | 编码率 | 必须在 [0.1, 0.95] 范围 | | MCS等级 | 必须在 [0, 28] 范围 | | HARQ最大重传 | 必须在 [0, 8] 范围 | | 发射功率 | 必须在 [0, 30] dBm 范围 | | 导频密度 | 必须在 [low, medium, high] 中 | | 调度器 | 必须在预定义的策略中 | | 功率控制 | 必须在预定义的策略中 | | 发射功率上限 | 不能超过任务指定的最大值 |

软约束（警告但允许通过）： | 条件 | 警告内容 | |-----|-----| | 高可靠性任务用了64QAM | 可能不够可靠 | | 低时延任务HARQ≥4次 | 可能增加时延 | | 节能任务发射功率>20dBm | 功耗偏高 | | 高吞吐任务用了BPSK | 吞吐可能不达标 |

6.3 链路级仿真算法

代码位置: [uran/twin/link_simulator.py](file:///e:/AI RAN Demo/uran/twin/link_simulator.py)

算法功能

比 KPI 模型更精细的链路级仿真，综合考虑增益、惩罚、编码时延和能效，模拟方案在指定环境下的实际表现。

输入

- plan : 通信方案
- environment : 环境参数 (SINR、干扰等级等)

输出

参数	说明
effective_sinr_db	综合处理后的有效SINR
bler	估算误块率
throughput_kbps	估算吞吐量
latency_ms	估算时延
spectral_efficiency	频谱效率
energy_cost	能耗估计

核心实现思路：增益/惩罚叠加模型

```
有效SINR = 环境SINR
            - 频率惩罚
            + 编码增益 (查表)
            + 导频增益 (查表)
            + HARQ增益 = HARQ重传 × 1.2 dB
            + 功率增益 = clamp((功率 - 10) / 5, 0, 4) dB
            - 调制灵敏度惩罚 (查表)
            + 干扰惩罚 (查表)
```

然后通过 Sigmoid 计算 BLER:

```
BLER = 1 / (1 + e^((有效SINR - 调制阈值) / 2))
BLER = clamp(BLER, 0.001, 0.999)
```

吞吐量、时延、能耗的计算类似 KPI 模型但包含了更多细节参数 (如编码时延表、调度器时延表等)。

能耗计算公式

基准能量 = $10^{(\text{发射功率}/10)} / 1000$
HARQ惩罚 = $1 + 0.05 \times \text{HARQ重传次数}$
总能耗 = 基准能量 $\times$ HARQ惩罚

能耗基于通信理论中的功率-能量关系，考虑了HARQ重传带来的额外能量开销。

6.4 故障注入算法

代码位置: [uran/twin/fault_injection.py](file:///e:/AI RAN Demo/uran/twin/fault_injection.py)

算法功能

在数字孪生环境中模拟各种异常情况，测试方案在恶劣条件下的**鲁棒性**——真正好的方案不仅正常环境下表现好，故障条件下也应该保持可接受的性能。

输入

- `environment` : 正常环境参数
- `fault_type` : 故障类型

输出

修改后的环境参数。

核心实现思路：5种故障类型

故障类型	修改操作	模拟场景
sudden_snr_drop	SNR和SINR各减10 dB	突发信号严重衰减
burst_interference	SINR减5 dB，干扰等级设为high	突发强干扰
high_mobility	多普勒频移 $\times 3$ ，SINR减2 dB	高速移动（高铁、车载）
power_limit	SINR减3 dB	功率受限（低电量模式）
module_failure	SINR减8 dB	通信模块故障

故障注入是数字孪生的核心优势——在虚拟环境中“搞破坏”成本为零，可以发现真实部署中可能遇到的隐患。

6.5 综合评分算法

代码位置: [uran/twin/scoring.py](file:///e:/AI RAN Demo/uran/twin/scoring.py)

算法功能

将链路仿真的多维KPI综合为一个**单一分数 (0~100)**，用于方案比较和决策。分数计算考虑了用户任务的优先级——不同任务看重不同的指标。

输入

- `metrics` : 正常环境的链路仿真KPI
- `intent` : 任务需求 (含优先级)
- `fault_metrics` : 故障注入后的KPI列表

输出

分值	说明
<code>reliability_score</code>	可靠性得分 (0~100)
<code>throughput_score</code>	吞吐量得分 (0~100)
<code>latency_score</code>	时延得分 (0~100)
<code>energy_score</code>	能效得分 (0~100)
<code>robustness_score</code>	鲁棒性得分 (0~100)
<code>final_score</code>	加权综合得分 (0~100)

核心实现思路：加权多目标评分

子评分公式：

可靠性得分 = $\min(100, 100 \times \text{目标BLER} / \text{实际BLER})$

吞吐量得分 = $\min(100, 100 \times \text{实际吞吐} / \text{目标吞吐})$

时延得分 = $\min(100, 100 \times \text{目标时延} / \text{实际时延})$

能效得分 = $\max(0, \min(100, 100 - \text{能耗} \times 10))$

鲁棒性得分 = $0.6 \times \text{故障下BLER指标} + 0.4 \times \text{故障下吞吐保持率}$

加权重 (根据任务优先级动态变化)：

任务优先级	可靠性	吞吐量	时延	能耗	鲁棒性
reliability	40%	15%	15%	10%	20%
throughput	15%	45%	15%	10%	15%
latency	20%	15%	40%	10%	15%
energy	20%	15%	10%	40%	15%
balanced	25%	25%	20%	10%	20%
anti_jamming	30%	15%	15%	5%	35%

最终得分 = $\sum$ (各子得分 × 对应权重)

这个算法的精妙之处在于：**不同用户看重的指标不同**——高吞吐任务的用户主要看吞吐量（权重45%），而抗干扰任务的用户则特别关注鲁棒性（权重35%）。

6.6 沙盒决策算法

代码位置: [uran/twin/sandbox.py](file:///e:/AI RAN Demo/uran/twin/sandbox.py) — `DigitalTwinSandbox.validate()`

算法功能

汇总五步验证的所有结果，做出最终决策：**ACCEPT（接受）**、**NEED_REWISE（需修改）**还是**REJECT（拒绝）**。

决策逻辑

链路仿真通过条件：

- $BLER \leq \max(\text{目标}BLER \times 3, 0.05)$ 且
- 时延 $\leq \text{目标时延} \times 2$

故障测试通过条件：

- 鲁棒性得分 ≥ 40

最终决策：

如果 最终得分 ≥ 75 且 链路仿真通过 且 故障测试通过：

→ ACCEPT（接受）

否则如果 最终得分 ≥ 60 ：

→ NEED_REVISE（需要修改但不算失败）

否则：

→ REJECT（拒绝）

第七部分：自演进与优化算法

7.1 进化循环算法

代码位置: [uran/evolution/evolution_loop.py](file:///e:/AI RAN Demo/uran/evolution/evolution_loop.py)

算法功能

实现系统的**自演进闭环**：AI生成方案 → 数字孪生验证 → 评分选择最优 → 运行时执行 → 监控KPI → 分析瓶颈 → 优化推荐 → 下一轮迭代。

核心实现思路

单次进化循环（run_once）

1. AI编配 → 生成意图+环境+候选方案
2. 数字孪生 → 逐个验证所有候选方案
3. 择优 → 选择ACCEPT中得分最高的方案
4. 微内核 → 加载方案并运行50步
5. 监控 → 收集运行时KPI
6. 优化推荐 → 基于KPI分析瓶颈
7. 经验存储 → 记录到DataLake
8. 策略记忆 → 更新最佳策略

多轮迭代

```
run_iterations(n=3):
    for i in 1..n:
        result = run_once(current_intent)
        分析瓶颈:
            BLER超标 → 下一轮强调"增强可靠性"
            吞吐不足 → 下一轮强调"提高吞吐"
            时延超标 → 下一轮强调"降低时延"
            无瓶颈 → 下一轮强调"保持并优化"
```

每次迭代的反馈被注入到下一轮的意图描述中，形成**"发现问题→调整目标→重新优化"的闭环**。

7.2 启发式优化推荐算法

代码位置: [uran/evolution/optimizer.py](file:///e:/AI RAN Demo/uran/evolution/optimizer.py)

算法功能

根据运行时的实际KPI与任务目标的差距，生成具体的优化建议。这是一个**基于规则的启发式系统**，类似通信专家的诊断思路。

输入

- intent : 任务目标（含目标BLER、最小吞吐、最大时延等）
- plan : 当前使用的方案
- runtime_kpis : 实际测得的KPI

输出

- reasons : 分析出的问题原因
- changes : 分类的优化建议（increase_reliability / increase_throughput / reduce_latency / fine_tuning）
- expected_improvement : 预期改善描述

核心诊断逻辑

如果 实测BLER > 目标BLER：

- 原因："BLER超标"
- 建议：降低调制阶数、提高导频、增加HARQ、适度提高功率

如果 实测吞吐 < 目标吞吐：

- 原因："吞吐不足"
- 建议：提高调制、提高编码率、降低导频和HARQ开销

如果 实测时延 > 目标时延：

- 原因："时延超标"
- 建议：减少HARQ次数、切换低时延调度器、选择低复杂度编码

如果 无问题：

- 建议：保持策略、小幅优化MCS、继续收集数据

这个算法体现了**通信专家的诊断经验**——知道每个问题对应哪些调整方向。例如BLER高，不能简单粗暴地降低调制，而是要从多个维度（调制、编码、HARQ、功率、导频）综合调整。

7.3 历史策略推荐算法

代码位置： [uran/evolution/recommender.py](file:///e:/AI RAN Demo/uran/evolution/recommender.py)

算法功能

从历史数据（DataLake）和策略记忆（StrategyMemory）中检索**针对特定场景类型的最优历史策略**，为新任务提供参考。

核心实现思路

1. 根据场景类型查询 DataLake，获取所有历史实验
2. 从 StrategyMemory 中获取该场景下得分最高的 N 条历史策略
3. 返回策略列表（含历史得分和推荐理由）
4. 如果历史数据不足，返回最近一条数据作为参考

这是一个**基于经验的推荐系统**——"以前在这个场景下什么方案效果最好，现在可以试试类似的"。

第八部分：频段配置算法

代码位置: [uran/common/band_config.py](file:///e:/AI RAN Demo/uran/common/band_config.py)

算法功能

管理系统的频段配置，计算频率对通信性能的影响。这是让系统能够模拟**真实5G频段**（如中国移动n41/n79）的核心。

预设频段

频段Key	名称	中心频率	带宽	适用场景
n41_100mhz	n41	2600 MHz	100 MHz	中国移动主力频段
n79_100mhz	n79	4900 MHz	100 MHz	高速热点覆盖
n41_50mhz	n41半带宽	2600 MHz	50 MHz	中等带宽
sim_default	仿真默认	2600 MHz	0.5 MHz	快速仿真验证

频率惩罚算法

频率惩罚 (dB) = $20 \times \log_{10}(\text{载波频率} / 2600)$

物理原理：根据 Friis 自由空间传播模型，信号衰减与频率的平方成正比。换算为 dB，即为 $20 \times \log_{10}(f/f_0)$ 。

实例计算：

- n41 (2600MHz)：惩罚 = $20 \times \log_{10}(2600/2600) = 0 \text{ dB}$ (基准)
- n79 (4900MHz)：惩罚 = $20 \times \log_{10}(4900/2600) \approx 5.5 \text{ dB}$

这意味着在其他条件相同时，n79 频段的信号会比 n41 弱约 5.5 dB，影响有效 SINR 进而影响 BLER 和吞吐量。

第九部分：波形生成与频谱分析算法

9.1 IQ 符号生成算法

代码位置: [uran/sdr/waveform.py](file:///e:/AI RAN Demo/uran/sdr/waveform.py) — `generate_iq_symbols()`

算法功能

生成与调制方式对应的 IQ (In-phase / Quadrature) 复数符号序列。IQ 是数字通信中将比特映射到电磁波的基本表示方式。

输入

参数	类型	说明
modulation	str	调制方式
n_symbols	int	生成的符号数 (默认1024)
seed	int	随机种子 (默认42)

输出

- 复数数组 (numpy complex128) , 每个元素代表一个调制符号

核心实现思路

BPSK (二进制相移键控) :

```
生成随机比特 b ∈ {0, 1}
符号 = 2b - 1 + 0j    (即 +1 或 -1)
```

星座点: {-1, +1}

QPSK (四进制相移键控) :

```
生成两路独立比特 bi, bq ∈ {0, 1}
符号 = [(2·bi - 1) + j(2·bq - 1)] / √2
```

星座点: 位于复平面45°、135°、225°、315°方向, 归一化能量

16QAM (16阶正交幅度调制) :

```
从 {-3, -1, 1, 3} 中随机选 I 路和 Q 路幅度
符号 = (I + jQ) / √10
```

归一化因子 $\sqrt{10} = \sqrt{(\text{平均能量})} = \sqrt{((3^2+1^2)\times 4/16)}$

64QAM (64阶正交幅度调制) :

从 $\{-7, -5, -3, -1, 1, 3, 5, 7\}$ 中随机选 I 路和 Q 路幅度
符号 = $(I + jQ) / \sqrt{42}$

归一化因子 $\sqrt{42} = \sqrt{\text{平均能量}}$

9.2 AWGN 噪声叠加算法

代码位置: [uran/sdr/waveform.py](file:///e:/AI RAN Demo/uran/sdr/waveform.py) — `add_awgn()`

算法功能

向信号附加上**加性白高斯噪声 (AWGN)** ——通信理论中最基本的噪声模型。

核心公式

信号功率 = 平均 $(|IQ\text{符号}|^2)$
噪声功率 = 信号功率 / $10^{(SNR/10)}$
噪声 = $\sqrt{\text{噪声功率}/2} \times [N(0,1) + j \cdot N(0,1)]$
输出信号 = 输入信号 + 噪声

物理含义: 信噪比 SNR (dB) 决定了噪声相对于信号的强度。SNR=10dB 意味着信号功率是噪声功率的10倍; SNR=0dB 意味着信号和噪声一样强。

9.3 功率谱密度估计算法

代码位置: [uran/sdr/spectrum.py](file:///e:/AI RAN Demo/uran/sdr/spectrum.py) — `estimate_psd()`

算法功能

使用FFT (**快速傅里叶变换**) + **Hanning窗**估计信号的功率谱密度 (Power Spectral Density, PSD), 可视化信号在频域的能量分布。

核心流程

1. 施加 Hanning 窗函数（减少频谱泄漏）
2. 执行 FFT 变换（时域 → 频域）
3. 计算幅度谱并转换为 dB 单位： $20 \times \log_{10}(|FFT|)$
4. 计算对应频率轴：`fftfreq(n, d=1/sample_rate)`
5. 执行 `fftshift` 将零频居中

这是**频谱分析**的基础工具——让你“看到”信号占用了哪些频率、频谱形状是什么。

9.4 BER 估计算法

代码位置: [uran/sdr/mock_sdr.py](file:///e:/AI RAN Demo/uran/sdr/mock_sdr.py) — `measure_metrics()`

算法功能

基于信噪比和调制方式，使用简化公式快速估算误码率（BER）及相关指标。

简化 BER 公式

调制方式	BER 近似公式
BPSK	$BER \approx 0.5 \times e^{(-SNR_linear)}$
QPSK	$BER \approx 0.7 \times e^{(-0.8 \times SNR_linear)}$
16QAM	$BER \approx e^{(-0.25 \times SNR_linear)}$
64QAM	$BER \approx e^{(-0.12 \times SNR_linear)}$

这些是通信理论中 AWGN 信道下各种调制方式 BER 的工程简化近似。

EVM 估算

$$EVM(\%) = 100 / \sqrt{SNR_linear}$$

EVM（误差向量幅度）衡量发射信号质量——EVM 越低越好。

丢包率估算

$$\text{丢包率} = 1 - (1 - BER)^{1000}$$

假设每包1000比特，任一比特出错就认为整个包丢失。

第十部分：实验运行器算法

代码位置: [uran/sim/experiment.py](file:///e:/AI RAN Demo/uran/sim/experiment.py)

算法功能

`ResearchExperimentRunner` 是整个仿真系统的**顶层调度器**。它将前述所有算法串联为一个完整的仿真流程。

单步仿真流程

```
step():  
    |— 更新时间 ( $time = step \times interval$ )  
    |— 更新节点位置 (含移动性)  
    |— 更新链路距离  
    |  
    |— 对每条活跃链路:  
    |   |— 标记链路已建立  
    |   |— 从信道时间线读取当前SNR/SINR  
    |   |— 获取上一轮BLER (用于链路自适应的惩罚)  
    |   |— 执行链路自适应决策 (→ 调制、编码等参数)  
    |   |— 更新链路状态  
    |   |— 执行KPI估算 (→ BLER、吞吐、时延等)  
    |   |— 记录KPI数据  
    |  
    |— step += 1
```

run_all() 全流程

```
循环执行 step() 直到 step == total_steps  
收集所有 KPI 记录  
返回完整的时间序列数据
```

干扰估计

如果有 jammer 类型节点在场：

干扰惩罚 = 3 dB

否则：

干扰惩罚 = 1 dB

附录：算法索引表

编号	算法名称	代码位置	所属模块	核心功能
1	调制模块	[modulation.py](file:///e:/AI RAN Demo/uran/modules/modulation.py)	通信模块	决定每符号比特数
2	信道编码模块	[coding.py](file:///e:/AI RAN Demo/uran/modules/coding.py)	通信模块	冗余纠错
3	HARQ重传模块	[harq.py](file:///e:/AI RAN Demo/uran/modules/harq.py)	通信模块	失败自动重传
4	导频模块	[pilot.py](file:///e:/AI RAN Demo/uran/modules/pilot.py)	通信模块	信道估计参考信号
5	调度器模块	[scheduler.py](file:///e:/AI RAN Demo/uran/modules/scheduler.py)	通信模块	资源分配策略
6	功率控制模块	[power_control.py](file:///e:/AI RAN Demo/uran/modules/power_control.py)	通信模块	发射功率调节
7	安全回退模块	[fallback.py](file:///e:/AI RAN Demo/uran/modules/fallback.py)	通信模块	异常安全保护

编号	算法名称	代码位置	所属模块	核心功能
8	链路自适应	[adaptation.py](file:///e:/AI RAN Demo/uran/sim/adaptation.py)	仿真核心	4档决策选择通信参数
9	KPI估算模型	[kpi_model.py](file:///e:/AI RAN Demo/uran/sim/kpi_model.py)	仿真核心	量化通信性能指标
10	信道时间线	[channel_timeline.py](file:///e:/AI RAN Demo/uran/sim/channel_timeline.py)	仿真核心	7种信道环境模拟
11	KPI记录器	[kpi_recorder.py](file:///e:/AI RAN Demo/uran/sim/kpi_recorder.py)	仿真核心	时序KPI数据管理
12	实验运行器	[experiment.py](file:///e:/AI RAN Demo/uran/sim/experiment.py)	仿真核心	仿真顶层调度
13	接口校验	[validators.py](file:///e:/AI RAN Demo/uran/twin/validators.py)	数字孪生	字段完整性和约束检查
14	约束校验	[validators.py](file:///e:/AI RAN Demo/uran/twin/validators.py)	数字孪生	参数范围与任务匹配检查

编号	算法名称	代码位置	所属模块	核心功能
15	链路级仿真	[link_simulator.py](file:///e:/AI RAN Demo/uran/twin/link_simulator.py)	数字孪生	增益/惩罚叠加仿真
16	故障注入	[fault_injection.py](file:///e:/AI RAN Demo/uran/twin/fault_injection.py)	数字孪生	5种异常模拟
17	综合评分	[scoring.py](file:///e:/AI RAN Demo/uran/twin/scoring.py)	数字孪生	加权多目标评分
18	沙盒决策	[sandbox.py](file:///e:/AI RAN Demo/uran/twin/sandbox.py)	数字孪生	ACCEPT/REJECT判定
19	进化循环	[evolution_loop.py](file:///e:/AI RAN Demo/uran/evolution/evolution_loop.py)	自演进	多轮迭代自优化
20	启发式优化	[optimizer.py](file:///e:/AI RAN Demo/uran/evolution/optimizer.py)	自演进	KPI对比规则推荐
21	历史推荐	[recommender.py](file:///e:/AI RAN Demo/uran/evolution/recommender.py)	自演进	历史最优策略检索
22	频段配置	[band_config.py](file:///e:/AI RAN Demo/uran/common/band_config.py)	公共工具	频率惩罚计算
23	IQ符号生成	[waveform.py](file:///e:/AI RAN Demo/uran/sdr/waveform.py)	SDR	BPSK/QPSK/QAM符号

编号	算法名称	代码位置	所属模块	核心功能
24	AWGN 噪声	[waveform.py](file:///e:/AI RAN Demo/uran/sdr/waveform.py)	SDR	加性白高斯噪声
25	功率谱估计	[spectrum.py](file:///e:/AI RAN Demo/uran/sdr/spectrum.py)	SDR	FFT频谱分析
26	BER估算	[mock_sdr.py](file:///e:/AI RAN Demo/uran/sdr/mock_sdr.py)	SDR	简化误码率公式

文档版本: Alpha 1.0

最后更新: 2026-05

维护者: younger025